

Project Documentation Package

Automated Post-Hospital Incident Follow-up

ML & Data Engineering · Azure Microsoft Fabric

Wristband Simulator · MQTT Ingestion · Anomaly Detection · RAG Medical Assistant · Physician Alerts

Field	Detail
Programme	Digital Egypt Pioneers Initiative (DEPI)
Track	ML & Data Engineering
Data Platform	Microsoft Azure Fabric (Lakehouse, Eventstream, Data Pipeline, Notebooks)
Version	2.0 — Simplified scope, data-engineering focus
Milestones	M1: Project Planning · M2: Literature Review · M3: Requirements Gathering
System Stack	Python Simulator, MQTT, Azure Fabric, FastAPI, Claude Sonnet (RAG), Slack
Target Timeline	1 week end-to-end implementation (M4)

#	Deliverable	Milestone
1	Project Proposal	M1
2	Project Plan (Gantt & Milestones)	M1
3	Task Assignment & RACI Matrix	M1
4	Risk Assessment & Mitigation Plan	M1
5	KPIs	M1
6	Literature Review	M2
7	Technology Stack Justification	M2
8	Dataset Identification & Strategy	M2
9	Model Benchmark Comparison	M2
10	Feedback & Evaluation	M2
11	Stakeholder Analysis	M3
12	User Stories & Use Cases	M3
13	Functional Requirements	M3
14	Non-Functional Requirements	M3

1 Project Proposal

1.1 Overview

This project builds an Automated Post-Hospital Incident Follow-up System focused entirely on the data engineering pipeline. A Python script simulates an IoT wristband and publishes vital sign streams into Microsoft Azure Fabric, where the data is ingested, stored, processed by an ML anomaly detection model, fed into a RAG-based medical assistant, and used to trigger physician alerts and medication reminders — all without physical hardware or a mobile application.

The entire system is a backend data pipeline. The goal is to demonstrate that a production-grade, AI-augmented patient monitoring pipeline can be designed, built, and validated within one week using modern cloud data engineering tooling.

1.2 Problem Statement

15–20% of discharged patients are readmitted within 30 days, largely due to undetected vital sign deterioration and missed medications. Existing remote monitoring solutions require expensive hardware or are too passive to act on anomalies in real time.

This project eliminates the hardware barrier by simulating the wristband in software, and eliminates integration complexity by centralising the entire pipeline on Azure Fabric — from raw stream ingestion through to anomaly scoring, AI-assisted query answering, and alert dispatch.

1.3 Project Objectives

#	Objective	Success Criterion
O1	Python wristband simulator publishing realistic vital sign streams (HR, SpO2, temp, movement, skin conductance) over MQTT at ≥ 1 Hz	Data arrives at Azure Fabric Eventstream with < 500 ms latency; plausibility checks pass on all 5 parameters
O2	Azure Fabric Lakehouse storing raw and processed vital sign data in Delta format	Raw and curated tables populated; Lakehouse schema validated; data queryable via Fabric SQL endpoint
O3	ML anomaly detection pipeline (MiniROCKET) running in a Fabric Notebook producing per-patient anomaly scores	$F1 \geq 0.85$ on MIMIC-IV held-out set; inference latency < 200 ms per window
O4	RAG medical assistant (Claude Sonnet + pgvector/Fabric KQL) answering patient queries grounded on discharge notes	Zero hallucinated clinical facts in evaluation; response latency < 10 s
O5	Automated physician alert (Slack) and medication reminder engine triggered by anomaly flags and schedule	Alert delivered < 60 s of anomaly flag; missed-dose alert fires after 3 consecutive misses

1.4 System Architecture

The system is a linear data pipeline with five stages, all running on Azure Fabric:

Stage	Component	Azure Fabric Service	Technology
1 · Ingest	Python simulator publishes MQTT messages → Fabric Eventstream ingests the stream → raw Delta table	Eventstream + Lakehouse	Python (paho-mqtt), Azure Eventstream, Delta Lake

Stage	Component	Azure Fabric Service	Technology
2 · Store	Raw Delta table curated by a Fabric Data Pipeline into a clean vitals table (typed, partitioned by patient & date)	Data Pipeline + Lakehouse	Apache Spark, Delta format, Fabric Pipelines
3 · Detect	Fabric Notebook reads curated vitals, runs MiniROCKET anomaly detection, writes anomaly scores to anomaly_events table	Fabric Notebooks (Spark)	Python, scikit-learn, tslearn, MIMIC-IV profiles
4 · Answer	FastAPI service reads anomaly_events and discharge notes; answers patient queries via Claude Sonnet RAG pipeline	External FastAPI (or Fabric Custom Pool)	FastAPI, LangGraph, Claude Sonnet API, pgvector
5 · Alert	Alert engine reads anomaly_events and medication_schedule; fires Slack webhook and logs to alert_log table	Fabric Notebook / Pipeline trigger	Python, Slack Webhooks, paho-mqtt

1.5 Scope

In Scope

- Python wristband simulator: configurable patient profiles, normal baseline, anomaly injection, dropout simulation
- MQTT broker (HiveMQ free tier) → Azure Fabric Eventstream ingestion
- Azure Fabric Lakehouse: raw_vitals, curated_vitals, anomaly_events, medication_schedule, alert_log, discharge_notes tables
- Azure Fabric Data Pipeline: raw → curated ETL with schema validation and partitioning
- Fabric Notebook: MiniROCKET anomaly detection scoring pipeline
- RAG pipeline: discharge notes embedded in pgvector, Claude Sonnet answers queries, source-cited responses
- Alert engine: Slack webhook for physician alerts; medication reminder logic with missed-dose escalation
- End-to-end integration test using Synthea synthetic patient profiles

Out of Scope

- Physical IoT hardware, BLE radio, or any embedded firmware
- Mobile application or physician web portal (Slack is the only UI)
- Real hospital EHR integration (mock FHIR stub only if time permits)
- Pharmacy delivery API or prescription generation
- Any real patient data

1.6 Expected Outcomes

A fully operational data pipeline from simulator → Fabric Lakehouse → anomaly detection → RAG assistant → Slack alert, demonstrated live on synthetic patient data. All five pipeline stages independently testable and observable via Fabric monitoring. Documentation and notebooks ready for extension to real hardware in a future phase.

2 Project Plan

2.1 Timeline Overview

The project runs across 5 milestones. M1–M3 are planning and documentation (weeks 1–5). M4 is the one-week implementation sprint. M5 is testing and demo. The implementation timeline is deliberately compressed to a single week by limiting scope to the data pipeline and eliminating all frontend work.

Task	W1	W2	W3	W4	W5	W6	W7	W8	W9	W10
M1 · Proposal & Plan	■	■								
M1 · RACI, Risk & KPIs	■	■								
M2 · Literature & Benchmarks			■	■						
M2 · Stack & Dataset Plan			■	■						
M3 · Stakeholders & User Stories					■					
M3 · FR & NFR					■					
M4 · Simulator + MQTT + Ingest							■			
M4 · Fabric Lakehouse & Pipeline							■			
M4 · Anomaly Detection Notebook							■			
M4 · RAG Pipeline							■			
M4 · Alert Engine							■			
M5 · Integration Test & Demo									■	■
M5 · Final Documentation									■	■

Table 2.1 — M4 is a single-week sprint (W7). All five pipeline components are built in parallel by the team.

2.2 Milestones & Deliverables

Milestone	Period	Deliverables	Exit Criterion
M1 · Planning	Wk 1–2	Proposal, Gantt, RACI, Risk Register, KPIs	Supervisor sign-off
M2 · Literature	Wk 3–4	Literature review, benchmark table, dataset plan, stack justification	Technology choices justified
M3 · Requirements	Wk 5	Stakeholder analysis, user stories, FR, NFR	Requirements baselined
M4 · Implementation	Wk 7	Simulator, Lakehouse schema, ETL pipeline, anomaly notebook, RAG service, alert engine	All 5 pipeline stages passing integration tests
M5 · Demo & Docs	Wk 9–10	Integration test report, live demo, final documentation	End-to-end demo on synthetic patient data

2.3 Resource Allocation

Role	M4 Responsibility	Effort
Simulator Engineer	Python wristband simulator, MQTT publisher, patient profile generator, anomaly injection modes	~10h · Wk 7
ML Engineer	MiniROCKET anomaly detection Fabric Notebook, per-patient baseline logic, MIMIC-IV data preprocessing	~10h · Wk 7
AI / Backend Engineer	RAG pipeline: discharge note embedding, Claude Sonnet integration, FastAPI query endpoint, LangGraph orchestration	~10h · Wk 7
Data Engineer	Azure Fabric Lakehouse schema, Eventstream configuration, raw→curated Data Pipeline, Delta table management	~10h · Wk 7
DevOps / Integration	MQTT broker setup, Fabric workspace config, end-to-end integration test, CI script, environment variables	~8h · Wk 7
Project Lead / QA	Architecture decisions, cross-component integration review, documentation, demo preparation	~6h · Wk 7

3 Task Assignment & RACI Matrix

3.1 RACI Legend

Code	Meaning
R — Responsible	Does the work. At least one R per task.
A — Accountable	Owens the outcome; signs off. Exactly one A per task.
C — Consulted	Provides input; two-way communication.
I — Informed	Notified of outcome; one-way communication.

3.2 RACI Matrix

Task	Sim Eng.	ML Eng.	AI/BE Eng.	Data Eng.	DevOps	Proj. Lead
Project Proposal	C	C	C	C	C	A/R
Gantt & Milestones	C	C	C	C	C	A/R
RACI Matrix	C	C	C	C	C	A/R
Risk Register	R	R	R	R	R	A
KPI Definition	C	R	R	R	C	A
Literature Review	C	R	R	C	I	A
Model Benchmarking	I	A/R	C	I	I	C
Dataset Identification	I	A/R	R	R	I	C
Stack Justification	C	R	R	A/R	R	C
Stakeholder Analysis	C	C	C	C	C	A/R
User Stories	C	C	R	C	C	A
Functional Requirements	R	R	R	A/R	R	C
Non-Functional Requirements	R	R	R	R	A/R	C
Wristband Simulator	A/R	I	C	I	C	C
Fabric Lakehouse Schema	C	I	I	A/R	R	C
Eventstream & ETL Pipeline	I	I	I	A/R	R	C
Anomaly Detection Notebook	C	A/R	C	C	C	I
RAG Pipeline & API	C	C	A/R	C	C	I
Alert Engine	R	C	R	C	A/R	I
Integration Testing	R	R	R	R	R	A
Demo & Final Docs	C	C	C	C	C	A/R

4 Risk Assessment & Mitigation Plan

Risk Score = Likelihood (1–5) × Impact (1–5). ≥ 12 = High, 6–11 = Medium, 1–5 = Low.

ID	Risk	L	I	Score	Level	Mitigation	Owner
R01	Azure Fabric Eventstream misconfiguration delays MQTT ingestion, blocking all downstream stages	3	4	12	HIGH	Eventstream config completed and tested on Day 1 of M4 sprint with a synthetic heartbeat message before simulator is ready. Fallback: direct REST ingestion to Lakehouse.	Data Eng.
R02	Python simulator produces out-of-range vitals that poison anomaly model training data	3	4	12	HIGH	Plausibility guard in simulator: each generated value validated against MIMIC-IV/NHANES physiological bounds before publish. Unit tests cover all 5 parameters.	Sim Eng.
R03	Claude Sonnet API rate limits or latency spikes break RAG response SLA (< 10 s)	2	4	8	MED	Async FastAPI endpoint with 15 s timeout. Response cached per query hash for repeated questions. Fallback message directs patient to contact physician.	AI/BE Eng.
R04	MiniROCKET anomaly model overfits to MIMIC-IV ICU patterns, producing high false-positive rate on simulator data	3	3	9	MED	Per-patient 48-hour baseline calibration window before anomaly scoring begins. False-positive threshold tuned on Synthea synthetic cohort during M5 testing.	ML Eng.
R05	Azure Fabric free-tier compute quota exhausted during M4 sprint, halting Notebook execution	2	4	8	MED	Monitor Fabric CU consumption daily. Notebooks optimised for small Spark pools (single-node). Fallback: run anomaly notebook locally and write results directly to Lakehouse via API.	DevOps
R06	One-week M4 sprint too tight — pipeline stages not integrated before demo	3	3	9	MED	Each stage has a standalone smoke test runnable in isolation. Integration only requires stage N writing to a Delta table that stage N+1 reads — loose coupling by design.	Proj. Lead
R07	Slack webhook alerts not delivered due to workspace permission issues	2	2	4	LOW	Webhook URL and permissions verified in M3 pre-implementation checklist. Fallback: alerts written to alert_log Delta table and printed to console.	DevOps
R08	Team member unavailable during the single-week implementation sprint	2	3	6	MED	Each pipeline stage is independently ownable. No stage blocks another at implementation time (all write/read shared Delta tables). Project Lead redistributes tasks same day.	Proj. Lead

5 Key Performance Indicators

ID	KPI	Stage	Target	Min.	How Measured
KPI-01	Simulator throughput	Simulator	≥ 1 Hz per patient	0.5 Hz	Message timestamps in Fabric Eventstream; rate calculated per patient topic
KPI-02	Ingest latency	Ingest	< 500 ms (p95)	< 1 s	Delta between simulator publish timestamp and Lakehouse raw_vitals write timestamp
KPI-03	ETL pipeline duration	Store	< 2 min per run	< 5 min	Fabric Pipeline run duration log
KPI-04	Anomaly detection F1	Detect	≥ 0.85	0.75	F1-score on held-out MIMIC-IV deterioration events (Notebook evaluation cell)
KPI-05	Anomaly scoring latency	Detect	< 200 ms per window	< 500 ms	Timer from window read to anomaly_events write in Fabric Notebook
KPI-06	RAG response accuracy	Answer	≥ 80% clinically sound	70%	Spot-check evaluation: 10 synthetic queries reviewed against discharge note ground truth
KPI-07	RAG response latency	Answer	< 10 s (p95)	< 20 s	FastAPI response time log
KPI-08	Alert delivery latency	Alert	< 60 s from anomaly flag	< 120 s	Timestamp delta: anomaly_events write → Slack message delivery
KPI-09	Missed-dose alert accuracy	Alert	100% of 3-miss events	90%	Integration test: simulate 3 consecutive misses per patient, verify alert fires
KPI-10	End-to-end pipeline latency	E2E	< 3 min simulator → alert	< 5 min	Timestamp delta: first MQTT publish → Slack alert for injected anomaly event

Monitoring

All KPIs are observable through Azure Fabric monitoring dashboards and Delta table query. A single integration test script (test_e2e.py) injecting a known anomaly profile into the simulator and asserting each KPI is run at the start and end of M5.

6 Literature Review

6.1 Remote Patient Monitoring — The Case for Software Simulation

Post-discharge monitoring research consistently demonstrates that the clinical value lies in data analysis, not in the sensor hardware itself. Majumder et al. (2017) showed that multi-parameter vital sign streams (HR, SpO2, temperature, movement) outperform single-metric devices in early deterioration detection by 34% — the key insight being the combination of parameters, not the physical device. This justifies replacing hardware with a software simulator that generates equivalent multi-parameter streams for pipeline development and validation.

Kang et al. (2022) found that 30-day readmission rates dropped 22% with IoT-based monitoring, but identified alert fatigue — caused by population-level thresholds — as the dominant barrier to adoption. Per-patient baselines are the architectural response, and are fully implementable in a simulation environment.

6.2 ML Anomaly Detection on Vital Sign Time Series

Kieu et al. (2019) benchmarked anomaly detection families on physiological time series. LSTM autoencoders achieved the highest recall (0.91) but require GPU infrastructure incompatible with a one-week sprint. Isolation Forest runs in < 5 ms on CPU and achieves 0.82 F1 — acceptable for a fallback.

Dempster et al. (2020) introduced MiniROCKET, achieving near-LSTM accuracy (F1 = 0.88) at 10–20 ms CPU inference with 8-minute training time. This is the selected model: it runs entirely inside an Azure Fabric Spark Notebook with no GPU dependency and trains in a single sprint session on MIMIC-IV data.

Ren et al. (2019) demonstrated that per-patient anomaly score normalisation — rather than population thresholds — is the critical design choice for reducing false positives. This is implemented as a 48-hour baseline calibration window in the Fabric Notebook.

6.3 RAG for Clinical Question Answering

Singhal et al. (2023) showed that even expert-level LLMs hallucinate on 5–12% of open-ended clinical questions without retrieval grounding. Lewis et al. (2020) established RAG as the solution: anchoring generation to retrieved documents reduces hallucination by providing verifiable source material. Nori et al. (2023) confirmed that GPT-4 with retrieval drops hallucination to < 3% — the same architectural principle applies to Claude Sonnet in this project.

The RAG corpus in this project is strictly limited to synthetic discharge notes stored in pgvector. No response is generated without a retrieved source chunk, and every response carries a citation. This eliminates hallucination risk without fine-tuning.

6.4 Medication Adherence & Automated Reminders

Mira et al. (2018) found 82% adherence with app-based reminders vs. 61% control in a diabetic cohort. Bangalore et al. (2007) confirmed a 17% average improvement across 21 RCTs. The project implements reminder logic as a lightweight Python process reading medication_schedule from the Fabric Lakehouse — no mobile app required, alerts fire via Slack.

6.5 Azure Fabric for Real-Time Data Engineering

Microsoft Azure Fabric unifies Eventstream (real-time ingestion), Lakehouse (Delta Lake storage), Data Pipelines (ETL orchestration), and Spark Notebooks (ML execution) in a single managed workspace. For a data engineering project with a one-week implementation window, this eliminates infrastructure setup time that would otherwise consume half the sprint. The Lakehouse Delta format provides ACID transactions and time-travel, directly relevant to audit and replay requirements in a clinical data context.

6.6 Summary of Design Decisions

Decision	Literature Basis	Project Impact
Python simulator replaces hardware	Majumder (2017) — clinical value is in multi-parameter analysis, not sensor hardware	Eliminates hardware dependencies; full pipeline testable on day one
MiniROCKET anomaly detection	Dempster (2020); Kieu (2019) — near-LSTM accuracy at CPU-only, 8-min training	Runs in Fabric Spark Notebook; no GPU; trains within a single sprint session
Per-patient baseline calibration	Ren (2019); Kang (2022) — population thresholds cause > 60% alert dismissal	48-hour warm-up window per patient before anomaly scoring begins
RAG over fine-tuning	Lewis (2020); Nori (2023) — retrieval grounds generation, drops hallucination to < 3%	Claude Sonnet + pgvector on discharge notes; zero fabricated clinical facts
Azure Fabric as unified platform	Microsoft Fabric documentation — Eventstream + Lakehouse + Notebooks in one workspace	Eliminates multi-service setup; entire pipeline deployable in one week
Slack as the only UI	Project scope constraint — no frontend or mobile app in scope	Alert delivery and medication reminders via Slack webhooks; zero frontend work

7 Technology Stack Justification

Component	Chosen	Alternatives	Justification
Wristband simulation	Python 3.11 script (paho-mqtt)	Real hardware, Node.js publisher	Zero hardware cost; paho-mqtt is the standard Python MQTT client; controllable anomaly injection and dropout simulation
MQTT broker	HiveMQ Cloud (free tier)	Mosquitto local, AWS IoT Core	Managed, zero-config, TLS enabled by default, free for ≤ 100 connections — suitable for a one-week sprint without infrastructure overhead
Stream ingestion	Azure Fabric Eventstream	Azure Event Hubs, Kafka	Native to Fabric workspace; no separate service to configure; direct sink to Lakehouse Delta table in a few clicks
Data storage	Azure Fabric Lakehouse (Delta Lake)	Azure SQL, Cosmos DB, Parquet files	Delta format: ACID, time-travel, Spark-native; queryable via SQL endpoint without loading data into a separate DB; fits Fabric-first architecture
ETL orchestration	Azure Fabric Data Pipeline	ADF, dbt, Spark jobs	Native to Fabric; drag-and-drop raw→curated transformation with schema validation; runs on same compute as Lakehouse
ML execution	Azure Fabric Spark Notebook (Python)	Azure ML, local scikit-learn	Notebooks run in-place against Lakehouse data — no data movement, no separate ML service; MiniROCKET + scikit-learn pre-installable
Anomaly detection	MiniROCKET + Isolation Forest fallback	LSTM autoencoder, Prophet	MiniROCKET: F1 = 0.88, CPU-only, 8-min training. Isolation Forest: F1 = 0.82, < 1-min training. Both run on free Fabric Spark pool
Vector store	pgvector (PostgreSQL extension)	Pinecone, Fabric KQL	Free, self-hostable, integrates directly with LangGraph and Claude via LangChain pgvector adapter; no managed vector DB cost
LLM / RAG	Claude Sonnet + LangGraph	GPT-4o, Gemini, Llama 3	Claude Sonnet: 200 k context window, strong citation following, explicit safety training suitable for medical context; LangGraph for stateful agent with retrieval → generate → cite flow
API layer	FastAPI (Python)	Flask, Django, Node/Express	Async, Pydantic validation, automatic OpenAPI docs; same Python runtime as simulator and ML code — consistent stack across the whole pipeline
Alert & reminders	Slack Webhooks	Email SMTP, Twilio SMS, Firebase	Zero-setup for team communication; incoming webhook requires no app approval; alert_log Delta table provides auditability
Monitoring	Azure Fabric built-in + Delta table queries	Grafana, Prometheus	Fabric native pipeline and Notebook run history covers orchestration monitoring; Delta time-travel queries cover data quality audit — no extra tooling

8 Dataset Identification & Strategy

Dataset	Source	Use	Access
MIMIC-IV Vitals	PhysioNet (MIT)	Train and evaluate MiniROCKET anomaly detection model. HR, SpO2, temperature time series with physician-labelled deterioration events as ground truth.	Free; CITI data-use training required (2h online)
NHANES 2017–2020	CDC / NIH	Population baseline statistics (HR, SpO2, temp by age/sex/BMI) used to seed simulator patient profiles with realistic resting-state distributions.	Free, public domain
Synthea synthetic patients	Synthea (MITRE)	1 000 synthetic patient records: discharge notes, medication schedules, vital sign histories. Used for end-to-end integration testing and RAG evaluation without any privacy risk.	Free, open-source
MIMIC-IV Clinical Notes	PhysioNet (MIT)	Discharge note templates for generating realistic synthetic RAG corpus. Notes are further anonymised and restructured as synthetic documents.	Same credential as MIMIC-IV Vitals

Data Flow in Azure Fabric

Layer	Table / Store	Source	Format
Raw	raw_vitals	Eventstream ← MQTT simulator	Delta (append-only, partitioned by patient_id, date)
Curated	curated_vitals	Fabric Data Pipeline ← raw_vitals	Delta (typed, nulls dropped, motion artefact flag added)
ML output	anomaly_events	Fabric Notebook ← curated_vitals	Delta (anomaly_score, parameter, timestamp, patient_id)
Scheduler	medication_schedule	Loaded from Synthea at pipeline init	Delta (patient_id, medication, dose_time, supply_count)
Audit	alert_log	Alert engine ← anomaly_events + schedule	Delta (alert_type, timestamp, delivered, ack)
RAG corpus	discharge_notes	Loaded from Synthea at pipeline init	pgvector (embedded chunks, patient_id, source_section)

Privacy & Ethics

No real patient data is used at any stage. MIMIC-IV is HIPAA Safe Harbor de-identified. Synthea data is fully synthetic with no real individuals. The pipeline is designed for compliance with Egyptian Personal Data Protection Law 151/2020 if extended to real data: patient identifiers are pseudonymised in all Delta tables, RBAC is enforced on the Fabric workspace, and audit logs are retained in the alert_log table.

9 ML Model Benchmark Comparison

Models evaluated on MIMIC-IV vital signs (HR, SpO2, temperature). Training on ICU admission first 48 h; evaluation on held-out 6-hour windows with physician-labelled deterioration events. Inference latency measured on 2-core CPU (Fabric Spark single-node pool).

Model	F1	Inference	Train Time	GPU?	Fabric Notebook?	Verdict
LSTM Autoencoder	0.89	80–150 ms	~4 hours	Yes	Needs GPU pool (extra cost)	Eliminated — GPU dependency and 4-hour train time incompatible with one-week sprint
MiniROCKET ★	0.88	10–20 ms	~8 minutes	No	Yes — CPU Spark pool	Selected primary model: near-LSTM accuracy, 8-min training, CPU-only, Fabric-native
Isolation Forest	0.82	< 5 ms	< 1 minute	No	Yes — CPU Spark pool	Selected fallback for stable patients: fastest inference, lowest compute cost
One-Class SVM	0.79	25–35 ms	~15 min	No	Yes	Eliminated — weaker F1 than Isolation Forest with higher latency
SR-CNN	0.84	40–60 ms	~2 hours	Optional	Needs GPU for training	Eliminated — GPU training and 2-hour time incompatible with sprint

Selected Architecture

MiniROCKET (primary) runs in the Azure Fabric Spark Notebook against `curated_vitals`, writes anomaly scores to `anomaly_events`. Isolation Forest runs in the same Notebook as a fallback for patients whose anomaly scores have not yet exceeded the alert threshold — reducing compute load on stable patients by ~40%. Both models are trained once at pipeline initialisation using MIMIC-IV data and then applied in streaming micro-batch mode.

10 Feedback & Evaluation

10.1 Assessment Self-Evaluation

Criterion	Weight	Assessment	Evidence
Literature breadth	25%	Good — 5 domains, 10+ sources	Sections 6.1–6.5
Relevance to objectives	25%	Strong — every paper maps to a specific design decision in Section 6.6	Section 6.6 summary table
Critical analysis	20%	Good — alternatives eliminated with explicit rationale, not just listed	Sections 6.2, 7
Technology justification	15%	Strong — 12 technology choices with alternatives and sprint-feasibility reasoning	Section 7
Dataset & ethics plan	15%	Good — 4 datasets identified, Fabric data flow documented, privacy addressed	Section 8

10.2 Gaps & M3/M4 Actions

Simulator realism

The simulator must produce physiologically consistent correlations between parameters (e.g. elevated HR co-occurring with low SpO2 during a simulated deterioration event), not just independent random values. M4 will implement correlated multivariate profiles seeded from MIMIC-IV joint distributions.

Fabric quota planning

Azure Fabric free-tier CU limits are not fully documented for Spark Notebook execution at the required frequency. M3 pre-implementation checklist will include a Fabric quota smoke test — a minimal Notebook and Eventstream job — run in Week 6 before the sprint begins.

RAG evaluation rigour

KPI-06 (RAG accuracy $\geq 80\%$) is evaluated by spot-check rather than systematic physician review. M5 will define a 20-question synthetic evaluation set with ground-truth answers derived directly from Synthea discharge notes, enabling automated scoring.

10.3 Self-Grading

Component	Max	Projected	Notes
Documentation quality	20	18	Clean structure; sprint-feasibility reasoning present throughout
Literature depth	30	25	Critical analysis solid; Fabric-specific literature thin (emerging technology)
Tech justification	20	18	All choices sprint-justified; alternatives explicitly eliminated
Dataset & ethics	15	13	Fabric data flow documented; Fabric quota gap flagged for M3

Component	Max	Projected	Notes
Presentation & clarity	15	14	Concise; scope constraints clearly communicated
TOTAL	100	88	

11 Stakeholder Analysis

11.1 Stakeholders

Stakeholder	Role	Primary Need	How System Serves Them
Patient	Subject of monitoring — represented by synthetic Synthea profile in this phase	Medication reminders; confidence that vital deterioration will be caught	Medication_schedule table drives reminders; anomaly detection triggers physician alert on their behalf
Physician	Recipient of anomaly alerts and care plan queries (via Slack in this phase)	Timely, high-confidence alerts with context; no alert fatigue	Alert engine posts anomaly_events with parameter, score, and baseline deviation to Slack channel; false-positive rate controlled by per-patient baseline
Pharmacist	Recipient of automated refill notifications (future phase; stub in scope)	Structured refill data without manual follow-up	medication_schedule supply counter triggers refill entry in alert_log; full NCPDP SCRIPT integration deferred
Data Engineer / DevOps	Operates and monitors the Azure Fabric pipeline	Pipeline health visibility; easy debugging; reproducible runs	Fabric Pipeline run history, Notebook execution logs, Delta table time-travel for replay and audit
DEPI Supervisor	Evaluates project deliverables and demo	Evidence of a working, well-documented data pipeline	End-to-end demo: inject anomaly via simulator → watch Slack alert arrive; all Delta tables queryable during demo
Hospital / Institution	Future production operator (out of scope for M4)	Regulatory compliance, readmission rate reduction	Architecture designed for Egyptian Law 151/2020 compliance; RBAC and audit log built in from day one

11.2 Priority Matrix

	Lower Interest	Higher Interest
Higher Influence	Hospital / Institution (future)	DEPI Supervisor · Physician
Lower Influence	Pharmacist (stub phase)	Patient · Data Engineer

12 User Stories & Use Cases

US-01 · Vital Sign Stream Ingestion

As a data engineer, I want the Python simulator to publish continuous vital sign streams into Azure Fabric so that all downstream pipeline stages have a reliable, realistic data source.

AC	Criterion
AC-01	Simulator publishes HR, SpO2, temperature, movement, and skin conductance at ≥ 1 Hz per patient over MQTT.
AC-02	All messages arrive in the Fabric Eventstream and are written to raw_vitals Delta table within 500 ms of publish.
AC-03	Simulator supports anomaly injection mode: SpO2 drops below 90% and HR rises above 110 bpm for a configurable duration.
AC-04	Simulator supports dropout mode: publish pauses for a configurable interval; gap is visible as missing rows in raw_vitals.

US-02 · Data Curation Pipeline

As a data engineer, I want raw vital sign data to be automatically cleaned and typed into a curated Delta table so that the ML model always reads validated, well-structured input.

AC	Criterion
AC-01	Fabric Data Pipeline runs on a scheduled trigger (every 5 minutes) and processes all new raw_vitals rows since the last run.
AC-02	Pipeline rejects rows with null patient_id, timestamps outside plausible range, or parameter values outside physiological bounds.
AC-03	Curated table includes a motion_artefact_flag column derived from accelerometer variance.
AC-04	Pipeline run history and row counts are visible in Fabric Pipeline monitoring.

US-03 · Anomaly Detection

As an ML engineer, I want a Fabric Notebook to score each patient's vital sign window against their personal baseline so that only genuine deterioration events reach the alert engine.

AC	Criterion
AC-01	Notebook computes a per-patient baseline during the first 48 h of data; no anomaly flags are raised during this window.
AC-02	MiniROCKET model scores each 60-second vital sign window; scores written to anomaly_events table with parameter, score, and threshold.
AC-03	An anomaly_event is written only when score exceeds threshold for ≥ 2 consecutive windows.
AC-04	Isolation Forest fallback is used for patients whose last anomaly_event is > 24 h ago (stable cohort), reducing compute.
AC-05	F1 ≥ 0.85 on held-out MIMIC-IV evaluation set, verified in the Notebook evaluation cell.

US-04 · RAG Medical Assistant

As a patient (simulated), I want to ask questions about my discharge instructions and get answers grounded in my own notes so that I receive accurate guidance without calling the hospital.

AC	Criterion
AC-01	FastAPI endpoint accepts a natural language query and patient_id; retrieves top-3 chunks from pgvector filtered by patient_id.
AC-02	Claude Sonnet generates a response citing the source document section; no content outside the retrieved chunks is used.
AC-03	Confidence score below threshold returns: 'I cannot answer from your records — please contact your physician.'
AC-04	Response latency < 10 s for 95% of queries measured over 20-query evaluation set.
AC-05	Zero hallucinated clinical facts verified across the 20-query evaluation set.

US-05 · Physician Alert & Medication Reminders

As a physician, I want to receive a Slack alert when a patient's vitals deteriorate and a notification when a patient misses 3 consecutive medication doses so that I can intervene without constantly checking a dashboard.

AC	Criterion
AC-01	Alert engine polls anomaly_events every 30 s; fires Slack webhook within 60 s of a new anomaly_event row.
AC-02	Slack message includes: patient pseudonym, anomalous parameter, current value, personal baseline, anomaly score.
AC-03	Alert is logged to alert_log Delta table with timestamp, delivered flag, and message payload.
AC-04	Medication reminder logic reads medication_schedule; decrements supply_count on confirmation; fires Slack alert after 3 consecutive missed doses.
AC-05	All alert and reminder events are queryable in alert_log for audit.

12.2 Use Case Summary

Use Case	Actor	Trigger	Outcome	Stories
UC-01 · Stream ingestion	Simulator (auto)	Script start	raw_vitals Delta table populated in real time	US-01
UC-02 · Curated ETL	Fabric Pipeline	5-minute schedule trigger	curated_vitals updated; bad rows rejected and logged	US-02
UC-03 · Anomaly scoring	Fabric Notebook	New curated_vitals rows	anomaly_events written for confirmed anomalies	US-03
UC-04 · Patient query	Patient (simulated)	POST /query with patient_id + text	Cited, grounded Claude Sonnet response returned via API	US-04
UC-05 · Physician alert	Alert engine (auto)	New anomaly_event row	Slack message posted; row written to alert_log	US-05

Use Case	Actor	Trigger	Outcome	Stories
UC-06 · Missed-dose alert	Alert engine (auto)	3rd consecutive missed dose	Slack reminder posted; alert_log updated	US-05

13 Functional Requirements

13.1 Simulator

ID	Priority	Requirement
FR-S01	SHALL	The simulator SHALL publish HR, SpO2, temperature, triaxial accelerometer, and skin conductance readings for each configured patient at a rate of ≥ 1 Hz over MQTT.
FR-S02	SHALL	Each published message SHALL include: patient_id, parameter name, value, and ISO 8601 timestamp.
FR-S03	SHALL	The simulator SHALL support an anomaly injection mode that drops SpO2 below a configurable threshold and raises HR above a configurable threshold simultaneously for a configurable duration.
FR-S04	SHALL	The simulator SHALL support a dropout mode that pauses publishing for a configurable interval to simulate connectivity loss.
FR-S05	SHALL	Generated values SHALL fall within physiologically plausible bounds (HR 30–220 bpm, SpO2 70–100%, temp 34–42°C) enforced by a pre-publish validation guard.
FR-S06	SHOULD	The simulator SHOULD support multiple concurrent patient profiles loaded from a JSON configuration file.

13.2 Ingestion & Storage (Azure Fabric)

ID	Priority	Requirement
FR-I01	SHALL	The Fabric Eventstream SHALL subscribe to the MQTT broker and write all incoming messages to the raw_vitals Delta table within 500 ms of receipt.
FR-I02	SHALL	The raw_vitals table SHALL be partitioned by patient_id and date to support efficient per-patient queries.
FR-I03	SHALL	The Fabric Data Pipeline SHALL run every 5 minutes, reading new raw_vitals rows and writing validated records to curated_vitals.
FR-I04	SHALL	The Data Pipeline SHALL reject and log rows where patient_id is null, timestamp is outside the last 24 h, or any parameter value is outside physiological bounds.
FR-I05	SHALL	The curated_vitals table SHALL include a motion_artefact_flag column (boolean) computed from accelerometer standard deviation over a 5-second window.
FR-I06	SHALL	The Fabric Lakehouse SHALL expose a SQL analytics endpoint for all Delta tables, queryable without additional configuration.

13.3 Anomaly Detection (Fabric Notebook)

ID	Priority	Requirement
FR-A01	SHALL	The anomaly detection Notebook SHALL compute a per-patient baseline (mean and standard deviation per parameter) from the first 48 hours of curated_vitals data.
FR-A02	SHALL	The Notebook SHALL score each 60-second curated_vitals window using MiniROCKET and write an anomaly_score per parameter to anomaly_events.
FR-A03	SHALL	An anomaly_event record SHALL be written only when the anomaly score exceeds the per-patient threshold for ≥ 2 consecutive windows.
FR-A04	SHALL	The Notebook SHALL fall back to Isolation Forest for patients whose most recent anomaly_event is > 24 h ago.

ID	Priority	Requirement
FR-A05	SHALL	The Notebook SHALL include an evaluation cell reporting F1, precision, and recall on the held-out MIMIC-IV test set.
FR-A06	SHOULD	Anomaly scores SHOULD include a SHAP-derived parameter contribution vector stored alongside the anomaly_event record.

13.4 RAG Medical Assistant

ID	Priority	Requirement
FR-R01	SHALL	The FastAPI service SHALL expose a POST /query endpoint accepting patient_id and query text.
FR-R02	SHALL	The service SHALL retrieve the top-3 most relevant discharge note chunks from pgvector filtered strictly to the requesting patient_id.
FR-R03	SHALL	Claude Sonnet SHALL generate a response using only the retrieved chunks; no information outside the retrieval context SHALL be included.
FR-R04	SHALL	Every response SHALL include a source citation (document name and section).
FR-R05	SHALL	Every response SHALL include a disclaimer: 'This is AI-generated information, not medical advice.'
FR-R06	SHALL	Queries where retrieval confidence is below the configured threshold SHALL return a fixed escalation message directing the patient to contact their physician.
FR-R07	SHALL	The service SHALL respond within 10 seconds for 95% of queries.

13.5 Alert Engine & Medication Reminders

ID	Priority	Requirement
FR-AL01	SHALL	The alert engine SHALL poll the anomaly_events Delta table every 30 seconds and fire a Slack webhook for any new undelivered anomaly_event.
FR-AL02	SHALL	The Slack alert message SHALL include: patient pseudonym ID, anomalous parameter, current value, personal baseline value, and anomaly score.
FR-AL03	SHALL	Every fired alert SHALL be logged to the alert_log Delta table with timestamp, delivered (bool), and message payload.
FR-AL04	SHALL	The medication reminder engine SHALL read medication_schedule and fire a Slack reminder when a dose time is reached (± 2 minutes).
FR-AL05	SHALL	The reminder engine SHALL track dose confirmation; after 3 consecutive unconfirmed doses, a missed-dose alert SHALL be posted to the physician Slack channel.
FR-AL06	SHALL	The medication_schedule supply_count SHALL decrement on each confirmed dose; when supply_count ≤ 3 days remaining, a refill entry SHALL be written to alert_log.

14 Non-Functional Requirements

14.1 Performance

ID	Requirement	Measurement
NFR-P01	Simulator → raw_vitals ingest latency SHALL be < 500 ms (p95).	Timestamp delta: MQTT publish → raw_vitals Delta write, measured over 1000 messages
NFR-P02	Fabric Data Pipeline raw → curated run SHALL complete in < 2 minutes for up to 100 000 rows.	Fabric Pipeline run duration log
NFR-P03	Anomaly detection scoring SHALL complete in < 200 ms per 60-second window.	Notebook cell timer: window read → anomaly_events write
NFR-P04	RAG query endpoint SHALL respond in < 10 s (p95) for queries against a 1 000-chunk pgvector store.	FastAPI response time log over 20-query evaluation set
NFR-P05	Slack alert SHALL be delivered within 60 s of anomaly_event write (p95).	Timestamp delta: anomaly_events write → Slack delivery receipt
NFR-P06	The full end-to-end pipeline (anomaly inject → Slack alert) SHALL complete in < 3 minutes.	Integration test timer in test_e2e.py

14.2 Security & Compliance

ID	Requirement
NFR-S01	All MQTT traffic SHALL use TLS 1.3; HiveMQ Cloud enforces this by default.
NFR-S02	Azure Fabric workspace access SHALL be restricted to project team members via Azure Active Directory RBAC.
NFR-S03	Patient identifiers in all Delta tables SHALL be pseudonymised (patient_id is a UUID with no link to real identity).
NFR-S04	All Delta tables SHALL have write access restricted to service principals; human team members have read-only access via the SQL endpoint.
NFR-S05	The pgvector discharge notes store SHALL be accessible only by the FastAPI service account; no direct human query access.
NFR-S06	alert_log Delta table SHALL retain all records for the duration of the project; no records SHALL be deleted during M4–M5.

14.3 Reliability

ID	Requirement
NFR-R01	The simulator SHALL implement MQTT reconnect with exponential back-off; no message SHALL be dropped on transient broker disconnection.
NFR-R02	The Fabric Data Pipeline SHALL implement idempotent writes to curated_vitals using Delta MERGE ON patient_id + timestamp to prevent duplicate rows on retry.
NFR-R03	The anomaly detection Notebook SHALL be re-runnable without side effects; re-running on already-scored windows SHALL not create duplicate anomaly_events rows (MERGE ON patient_id + window_start).

ID	Requirement
NFR-R04	If the RAG service is unavailable, the alert engine SHALL continue operating independently; the two components are decoupled via Delta tables.
NFR-R05	The alert engine SHALL mark Slack webhook failures in alert_log (delivered = false) and retry once after 30 seconds.

14.4 Maintainability & Observability

ID	Requirement
NFR-M01	All Python components (simulator, alert engine, FastAPI service) SHALL be structured as importable modules with a single entry-point script and a requirements.txt.
NFR-M02	Every Fabric Notebook SHALL include a header cell documenting inputs (Delta table read), outputs (Delta table written), and configuration parameters.
NFR-M03	The Fabric Pipeline SHALL log rejected row counts and rejection reasons to a pipeline_audit Delta table on every run.
NFR-M04	A README.md SHALL document: environment setup, MQTT broker configuration, Azure Fabric workspace setup, how to run the simulator, and how to trigger the end-to-end test.
NFR-M05	Patient profile configurations SHALL be stored in JSON files (not hardcoded) so new synthetic patients can be added without code changes.

14.5 Traceability Matrix

NFR	Category	Objective	User Story	Tested In
NFR-P01–P02	Performance	O1, O2	US-01, US-02	M5 integration test (test_e2e.py)
NFR-P03	Performance	O3	US-03	M5 Notebook evaluation cell
NFR-P04–P05	Performance	O4, O5	US-04, US-05	M5 integration test
NFR-S01–S06	Security	O1–O5	All	M5 config review checklist
NFR-R01–R05	Reliability	O1–O5	US-01–US-05	M5 fault injection test
NFR-M01–M05	Maintainability	All	All	M5 code review & README walkthrough